



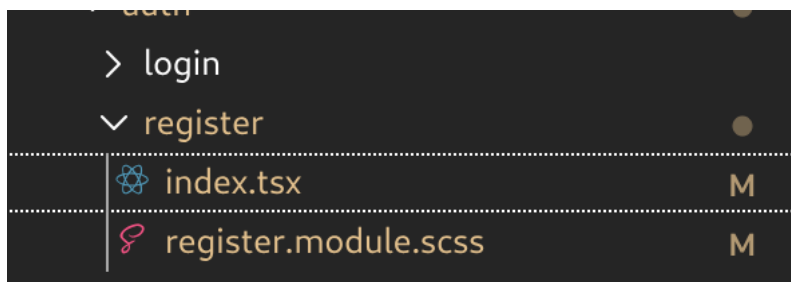
Mata Kuliah : Pemrograman Framework
Program Studi : D4 – Teknik Informatika
Semester : 6

Kelas : TI-3C
NIM : 2341720165
Nama : Fandy Wahyu Hanzura
Jobsheet Ke- : 14

Laporan Jobsheet

Praktikum 1

1.



```
1  import Link from "next/link";
2  import style from "@/views/auth/register/register.module.scss";
3
4  export default function TampilanRegister() {
5    return (
6      <div className={style.register}>
7        <h1 className={style.register_title}>Halaman Register</h1>
8        <div className={style.register_form}>
9
10         <div className={style.register_form_item}>
11           <label htmlFor="email" className={style.register_form_item_label}>
12             <input type="email" id="email" name="email" placeholder="Email" class
13           </div>
14
15         <div className={style.register_form_item}>
16           <label htmlFor="Fullname" className={style.register_form_item_labe
17             <input type="text" id="Fullname" name="Fullname" placeholder="Fullnam
18           </div>
19
20         <div className={style.register_form_item}>
21           <label htmlFor="Password" className={style.register_form_item_labe
22             <input type="password" id="Password" name="Password" placeholder="Pas
23           </div>
24
25         <button type="submit">Register</button>
26
27         <p className={style.register_link}>
28           Belum punya akun? </link href="/auth/Login">Ke Halaman Login</link>
```



```
src > pages > auth > register.tsx > ...  
1  import TampilanRegister from "@views/auth/register";  
2  
3  export default function HalamanRegister(){  
4      return(  
5          <div>  
6              <TampilanRegister />  
7          </div>  
8      )  
9  }
```

```
src > views > auth > register > register.module.scss > ...  
1  .register {  
2      min-height: 100vh;  
3      display: flex;  
4      flex-direction: column;  
5      align-items: center;  
6      justify-content: center;  
7      background: linear-gradient(135deg, #1e3c72, #2a5298);  
8      padding: 20px;  
9  
10     &__title {  
11         width: 100%;  
12         text-align: center;  
13         font-size: 36px;  
14         font-weight: 700;  
15         color: #ffffff;  
16         margin-bottom: 40px;  
17         animation: fadeIn 0.8s ease;  
18     }  
19  
20     &__form {  
21         background: #ffffff;  
22         width: 100%;  
23         max-width: 420px;  
24         padding: 40px;  
25         border-radius: 16px;  
26         box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15);  
27         animation: fadeIn 1s ease;  
28     }
```



Halaman Register

Email

Fullname

Password

Register

[Belum punya akun? Ke Halaman Login](#)

- Membuat halaman registrasi pada auth register

```
32 export async function signUp(userData: any, callback: Function) {  
33   try {  
34     const q = query(collection(db, "users"), where("email", "=", userData.email));  
35     const snapshot = await getDocs(q);  
36  
37     if (!snapshot.empty) {  
38       return callback({ status: "error", message: "User already exists" });  
39     }  
40  
41     await addDoc(collection(db, "users"), userData);  
42     callback({ status: "success", message: "User registered successfully" });  
43   } catch (error: any) {  
44     callback({ status: "error", message: error.message });  
45   }  
46 }  
47 }
```

2.



```
1 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2 import { signUp } from '@utils/db/servicefirebase'
3 import type { NextApiRequest, NextApiResponse } from 'next'
4
5 type Data = {
6   name: string
7   alamat: string
8 }
9
10 export default async function handler(
11   req: NextApiRequest,
12   res: NextApiResponse<Data>
13 ) {
14   if (req.method === "POST") {
15     await signUp(req.body, (result: { status: string, message: string }) => {
16       if (result.status === "success") {
17         res.status(200).json({ name: result.message, alamat: "" });
18       } else {
19         res.status(400).json({ name: result.message, alamat: "" });
20       }
21     });
22   } else {
23     res.status(405).json({ name: "Method not allowed", alamat: "" });
24   }
25 }
```

```
6 export default function TampilanRegister() {
7   const [isLoading, setIsLoading] = useState(false);
8   const [error, setError] = useState("");
9   const { push } = useRouter();
10
11   const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
12     event.preventDefault();
13     setError("");
14     setIsLoading(true);
15
16     const form = event.currentTarget;
17     const formData = new FormData(form);
18     const data = {
19       email: formData.get("email") as string,
20       fullname: formData.get("Fullname") as string,
21       password: formData.get("Password") as string,
22     };
23
24     const response = await fetch("/api/register", {
25       method: "POST",
26       headers: {
27         "Content-Type": "application/json",
28       },
29       body: JSON.stringify(data),
30     });
31   }
```



Halaman Register

Email

Kremlin@gmail.com

Fullname

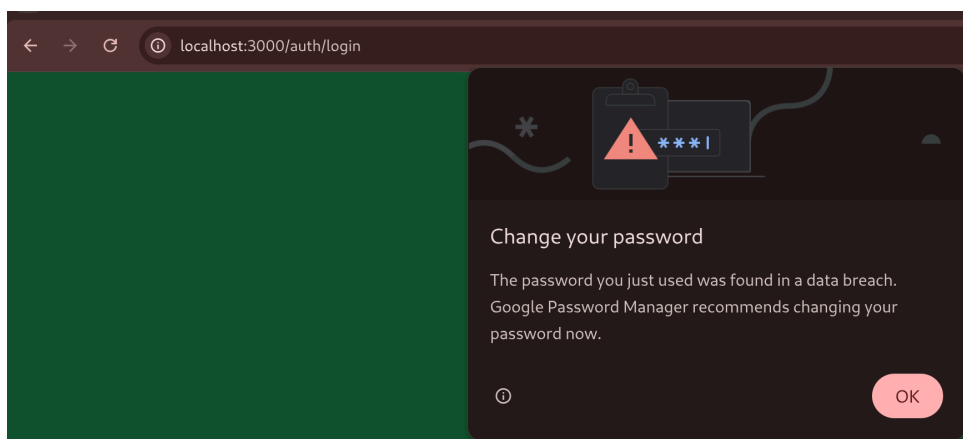
krav maz

Password

....

Register

Sudah punya akun? Ke Halaman Login



- Membuat fungsi signup pada file firebase service dan membuat file api untuk sign up



3.

```
Welcome to fish, the friendly interactive shell
Type help for instructions on how to use fish

...git\PemrogramanFramework\pertemuan02\my-app → ( main) 22.22.1 0ms 5:27 PM
> fandy >> npm install bcrypt --force
npm WARN using --force Recommended protections disabled.

added 3 packages, and audited 495 packages in 1s

162 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
...PemrogramanFramework\pertemuan02\my-app → ( main) 22.22.1 1.576s 5:27 PM
> fandy >> npm install --save-dev types/bcrypt --force
npm WARN using --force Recommended protections disabled.

up to date, audited 495 packages in 866ms

162 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
...PemrogramanFramework\pertemuan02\my-app → ( main) 22.22.1 1.034s 5:28 PM
> fandy >>
```

```
5 export async function signUp(
6   userData: { email: string; fullname: string; password: string; role?: string },
7   callback: Function
8 ) {
9   try {
10
11     const q = query(collection(db, "users"), where("email", "==", userData.email));
12     const snapshot = await getDocs(q);
13
14     if (!snapshot.empty) {
15       return callback({ status: "error", message: "User already exists" });
16     }
17
18     userData.password = await bcrypt.hash(userData.password, 10);
19     userData.role = "user";
20
21     await addDoc(collection(db, "users"), userData);
22     callback({ status: "success", message: "User registered successfully" });
23
24   } catch (error: any) {
25     callback({ status: "error", message: error.message });
26   }
27 }
```



Halaman Register

Email

amtrax@gmail.com

Fullname

amtrax

Password

.....

Register

Sudah punya akun? Ke Halaman Login

🏠 > users > jZ6STBsMW6H...			☁ More in Google Cloud ▾		
🏠 (default)	📁 users	⋮	📄 jZ6STBsMW6HGFmcMTI3B	⋮	
+ Start collection	+ Add document		+ Start collection		
products	4fwvIMjNbDnEQ1vb0fzC		+ Add field		
users >	jZ6STBsMW6HGFmcMTL3B >		email: "amtrax@gmail.com"		
			fullname: "amtrax"		
			password: "\$2b\$10\$OodOWGt1bER3AvLCIE69NO.1Wsac		
			role: "user"		



Halaman Register

User already exists

Email

amtrax@gmail.com

Fullname

amtrax

Password

.....

Register

Sudah punya akun? Ke Halaman Login

- Menginstall crypt agar password terenkripsi dan membuat gate check agar tidak terjadi redundansi data

Tugas praktikum

1.

```
<div className={style.register_form_item}>
  <label htmlFor="role" className={style.register_form_item_label}>Role</label>
  <select name="role" id="role" className={style.register_form_item_input}>
    <option value="member">User</option>
    <option value="admin">Admin</option>
  </select>
</div>
```



```
export async function signUp({
  userData: { email: string; fullname: string; password: string; role?: string },
  callback: Function
}) {
  try {
    const q = query(collection(db, "users"), where("email", "==", userData.email));
    const snapshot = await getDocs(q);

    if (!snapshot.empty) {
      return callback({ status: "error", message: "User already exists" });
    }

    userData.password = await bcrypt.hash(userData.password, 10);
    userData.role = userData.role || "member";
  }
}
```

The screenshot shows a registration form with the following fields: Email (amtrax1@gmail.com), Fullname (amtrax1), Password (masked with dots), Role (User), and a Register button. A red error message "User already exists" is displayed at the top of the form. Below the form, there is a link: "Sudah punya akun? Ke Halaman Login".

+ Add field

```
email: "amtrax1@gmail.com"
fullname: "amtrax1"
password: "$2b$10$Da9Q3CwSvjVpemW9dn6rO/JMH5f"
role: "member"
```



- Menambahkan handler di front dan di api untuk verifikasi email dan password dengan jumlah karakter tertentu lalu ditambahkan pula role verifikasi pada frontend dan backend

Pertanyaan refleksi

1. Mengapa password harus di-hash?

- Password harus melalui proses *hashing* untuk menjaga **kerahasiaan data pengguna (Data Privacy)**. Dengan menggunakan algoritma seperti **bcrypt**, password teks polos (*plain text*) diubah menjadi kode acak yang tidak dapat dikembalikan ke bentuk aslinya (*one-way function*). Hal ini bertujuan agar apabila database mengalami kebocoran, pihak yang tidak berwenang tetap tidak dapat mengetahui password asli pengguna, sehingga akun pengguna tetap terlindungi.

2. Apa perbedaan addDoc dan setDoc?

Perbedaan utama terletak pada **pengelolaan Document ID**:

- **addDoc()**: Digunakan untuk membuat dokumen baru di mana Cloud Firestore akan menghasilkan ID dokumen secara otomatis dan acak. Fungsi ini sangat efektif untuk data kolektif seperti log aktivitas atau postingan.
- **setDoc()**: Digunakan untuk membuat atau memperbarui dokumen dengan ID yang ditentukan sendiri oleh pengembang (misalnya menggunakan *Email* atau *UID* sebagai ID). Jika ID tersebut sudah ada, maka **setDoc** akan menimpa data yang lama kecuali ditambahkan opsi *merge*.

3. Mengapa perlu validasi method POST?

- Validasi terhadap method POST diperlukan untuk memastikan **keamanan dan integritas endpoint API**. Method POST dirancang untuk mengirimkan data sensitif melalui *request body*, bukan melalui URL seperti method GET. Dengan membatasi akses hanya melalui



POST, pengembang dapat mencegah percobaan pengiriman data ilegal melalui URL browser dan memastikan bahwa permintaan data hanya datang dari form aplikasi yang sah.

4. Apa risiko jika email tidak dicek unik?

- Tidak melakukan pengecekan keunikan email akan menyebabkan **redundansi data dan inkonsistensi autentikasi**. Risiko teknisnya adalah satu alamat email dapat terasosiasi dengan banyak akun berbeda. Hal ini akan membingungkan sistem saat proses *Login* atau *Reset Password*, karena sistem tidak dapat menentukan akun mana yang harus divalidasi, sehingga merusak alur logika sistem keamanan secara keseluruhan.

5. Apa fungsi role pada user?

- Atribut **role** berfungsi sebagai instrumen **Otorisasi (Role-Based Access Control / RBAC)**. Atribut ini digunakan untuk membedakan hak akses dan wewenang antar pengguna di dalam aplikasi. Misalnya, pengguna dengan role **admin** memiliki otoritas untuk mengelola data produk dan transaksi, sedangkan role **member** hanya diberikan izin untuk melihat katalog dan melakukan pembelian. Tanpa adanya **role**, sistem tidak memiliki batasan kendali akses bagi penggunanya.